Introducción

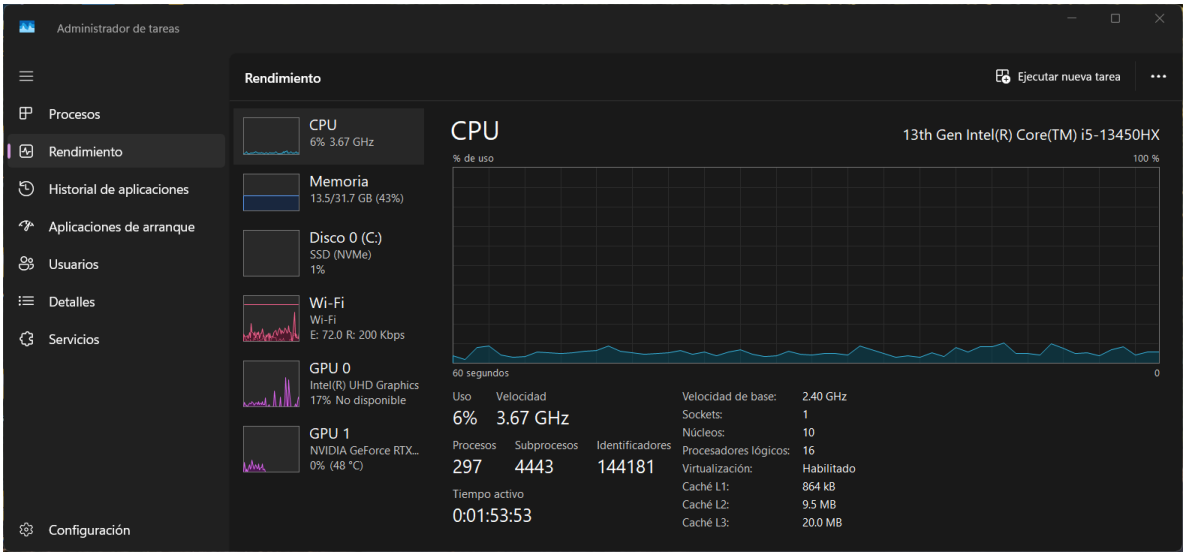
En esta práctica seguí la guía del README.md del proyecto **biblioteca-api** (github.com/carrenoutb/biblioteca-api), una API REST en Go + Gin + MySQL con arquitectura por capas (handler → service → repository). El objetivo fue levantar la API en mi computador con **Windows** usando solo Docker, sin instalar Go ni MySQL, y verificar que funciona. Como Docker necesita un kernel de Linux, el proceso incluyó instalar WSL 2 con Ubuntu antes de Docker Desktop. El informe narra el proceso fase por fase, con las evidencias de cada paso, y responde las preguntas de comprensión conectándolas con la investigación de la Parte 1.

Paso 1: Instalación de Docker en Windows

Trabajé en **Windows**. Antes de instalar, es importante entender por qué Docker necesita Linux. Como vimos en la Parte 1, un contenedor no trae su propio sistema operativo: comparte el **kernel** del host y usa funciones propias del kernel de Linux (*namespaces* y *cgroups*) para aislar procesos. Windows no tiene un kernel de Linux, así que Docker Desktop lo obtiene a través de **WSL 2** (*Windows Subsystem for Linux*), que ejecuta un kernel de Linux real dentro de una máquina virtual muy ligera administrada por Windows. Por eso el primer paso fue instalar WSL 2 con Ubuntu, y después Docker Desktop.

Verificar que la virtualización esté habilitada

WSL 2 necesita que la virtualización del procesador esté activa. Lo comprobé en el **Administrador de tareas** → **Rendimiento** → **CPU**, donde aparece *Virtualización: Habilitado*. Si apareciera deshabilitada, habría que activarla en la BIOS/UEFI del computador (opción *Intel VT-x* o *AMD-V/SVM*).

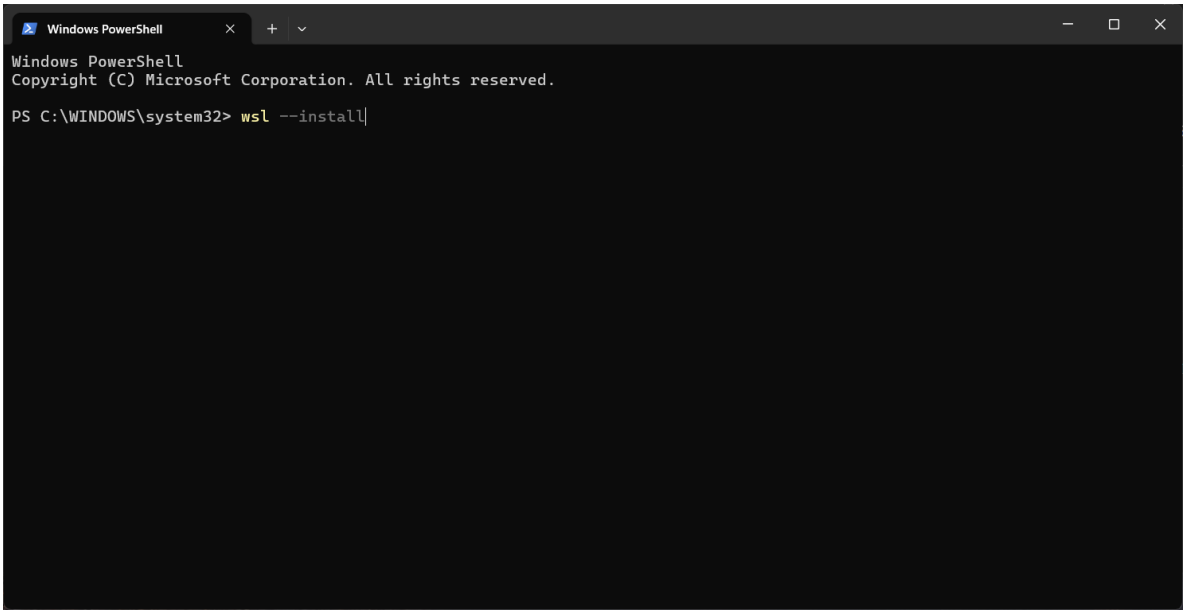


Instalar WSL 2 y Ubuntu

Abrí **PowerShell como administrador** (clic derecho en el menú Inicio → *Terminal (Administrador)*) y ejecuté:

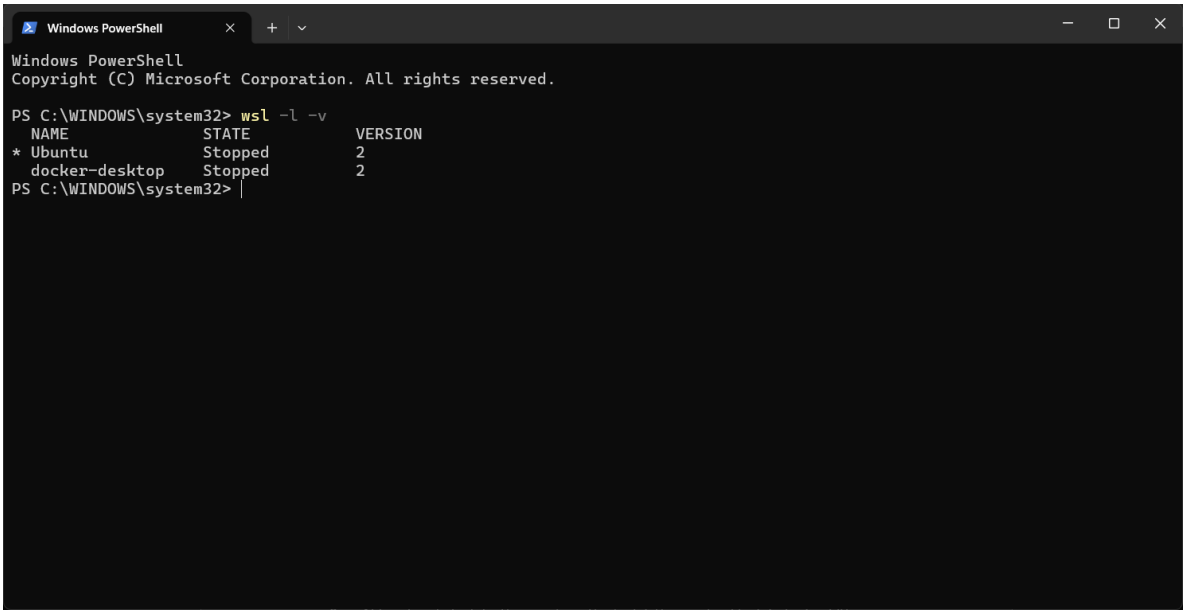
```
wsl --install
```

Este comando habilita las características de Windows necesarias (Subsistema de Windows para Linux y Plataforma de máquina virtual), descarga el kernel de Linux y **descarga e instala Ubuntu**, que es la distribución que WSL instala por defecto.



Al terminar, **reinicié el computador**. Para confirmar que Ubuntu quedó instalado y corriendo sobre WSL 2, ejecuté en PowerShell:

wsl -l -v



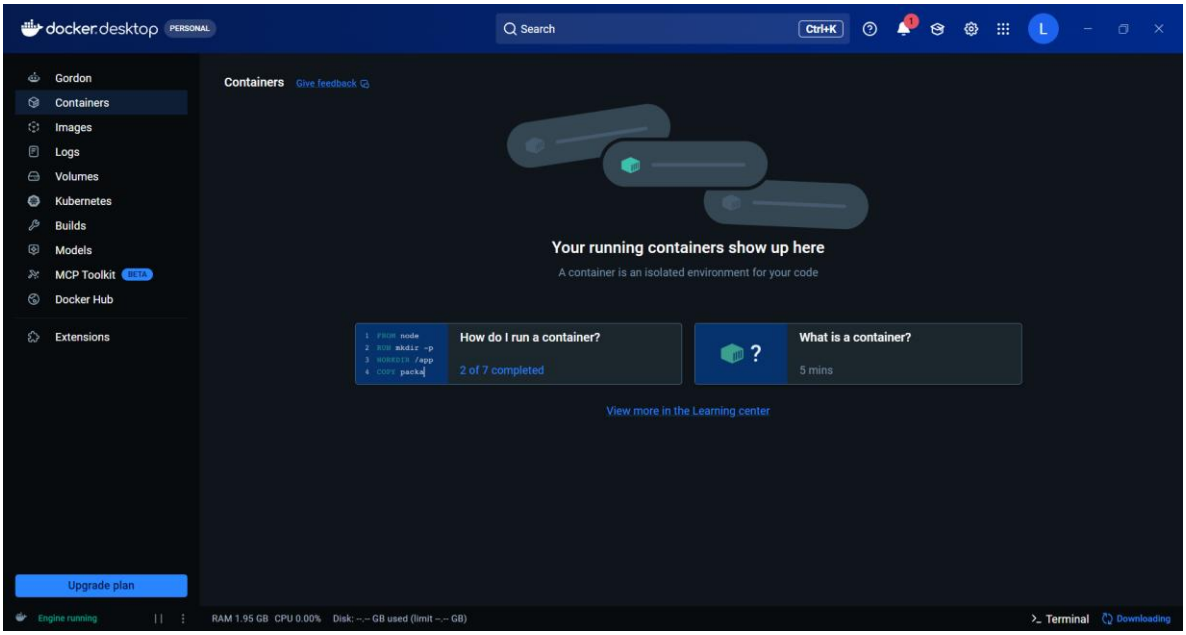
1.3 Descargar e instalar Docker Desktop

1. Entré a la página oficial de Docker (docker.com/products/docker-desktop) y descargué **Docker Desktop for Windows**.
2. Ejecuté el instalador y dejé marcada la opción **Use WSL 2 instead of Hyper-V**, para que Docker use el kernel de Linux de WSL 2 que instalé en el paso anterior.

3. Al terminar la instalación, cerré sesión / reinicié cuando el instalador lo pidió.

Abrir Docker Desktop

Abrí **Docker Desktop** desde el menú Inicio, acepté los términos de servicio y esperé a que en la esquina inferior izquierda apareciera **Engine running** (motor en ejecución). Ese indicador significa que el **daemon** (dockerd), el motor que vimos en la Parte 1, ya está activo dentro de WSL 2 y listo para recibir órdenes del cliente. Y por último hice log con mi correo.

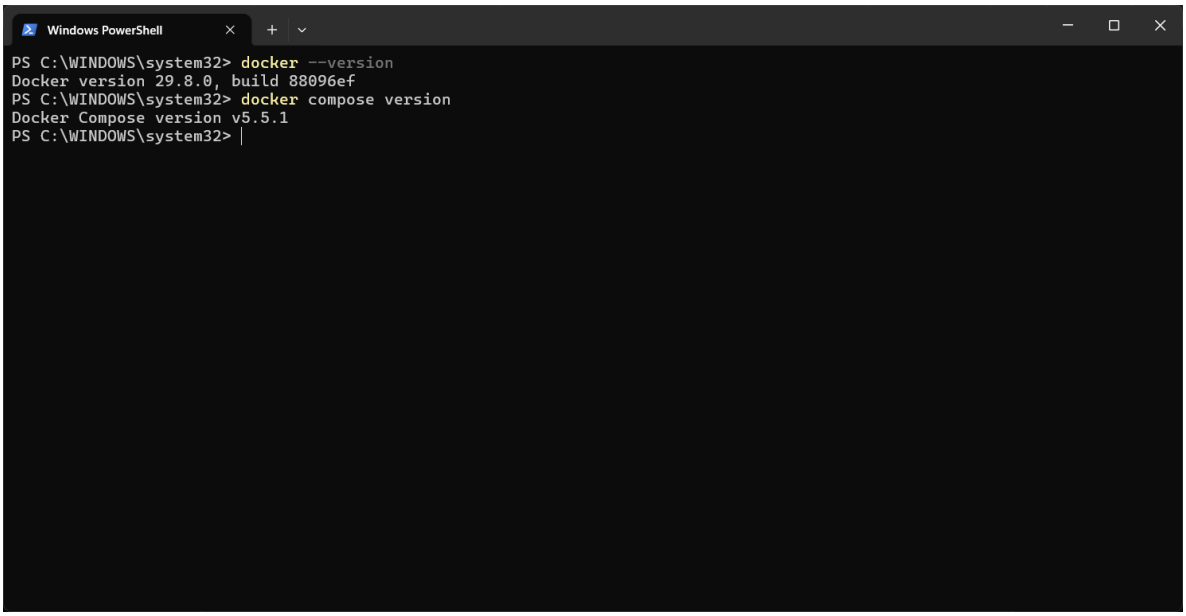


Verificar la instalación

En PowerShell ejecuté:

docker --version
docker compose version

Ambos comandos respondieron con su versión. Esto confirma que el cliente de Docker y el plugin de Compose están instalados.

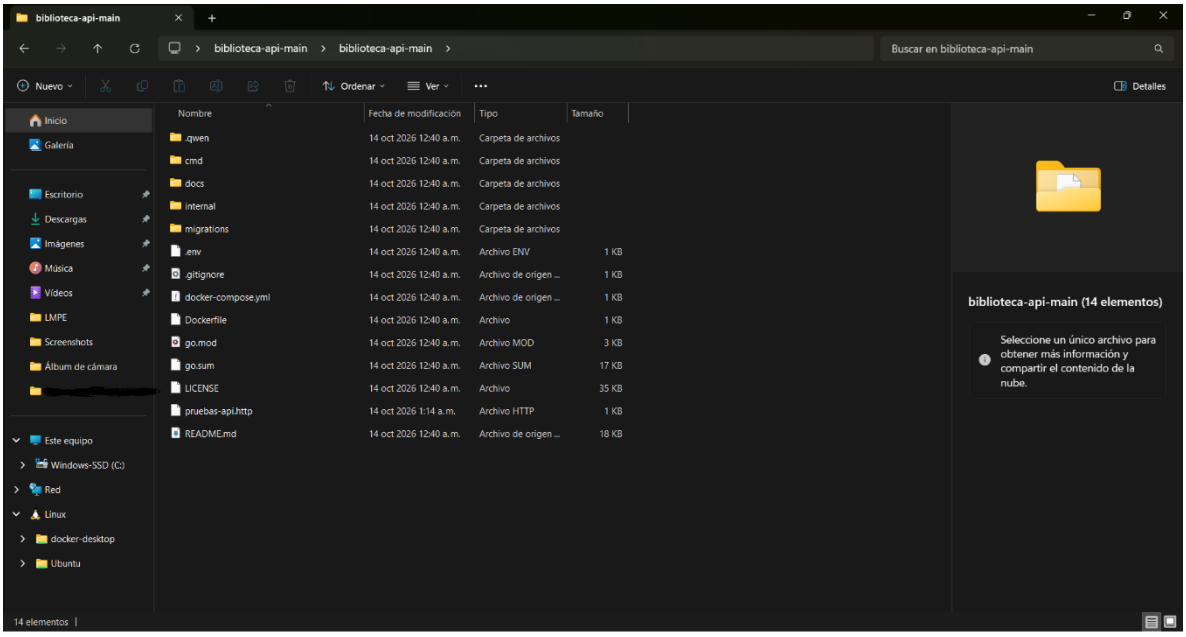


Paso 2: Puesta en marcha

Descargar el proyecto

descargué el proyecto como archivo ZIP:

1. Entré a github.com/carrenoutb/biblioteca-api.
2. Hice clic en el botón verde **Code** y luego en **Download ZIP**. Se descargó el archivo biblioteca-api-main.zip.
3. En el Explorador de archivos, hice clic derecho sobre el ZIP → **Extraer todo…** → **Extraer**. Se creó la carpeta biblioteca-api-main con todos los archivos del proyecto (Dockerfile, docker-compose.yml, .env.example, migrations/, etc.).



Después abrí una terminal dentro de la carpeta del proyecto: en el Explorador, dentro de la carpeta biblioteca-api-main (la que contiene el docker-compose.yml), clic derecho en un espacio vacío → **Abrir en Terminal**. Así todos los comandos siguientes se ejecutan en la carpeta correcta.

Al extraer el ZIP en Windows, a veces queda una carpeta dentro de otra con el mismo nombre (biblioteca-api-main\biblioteca-api-main). La correcta es la que contiene directamente el docker-compose.yml.

Las piezas del proyecto

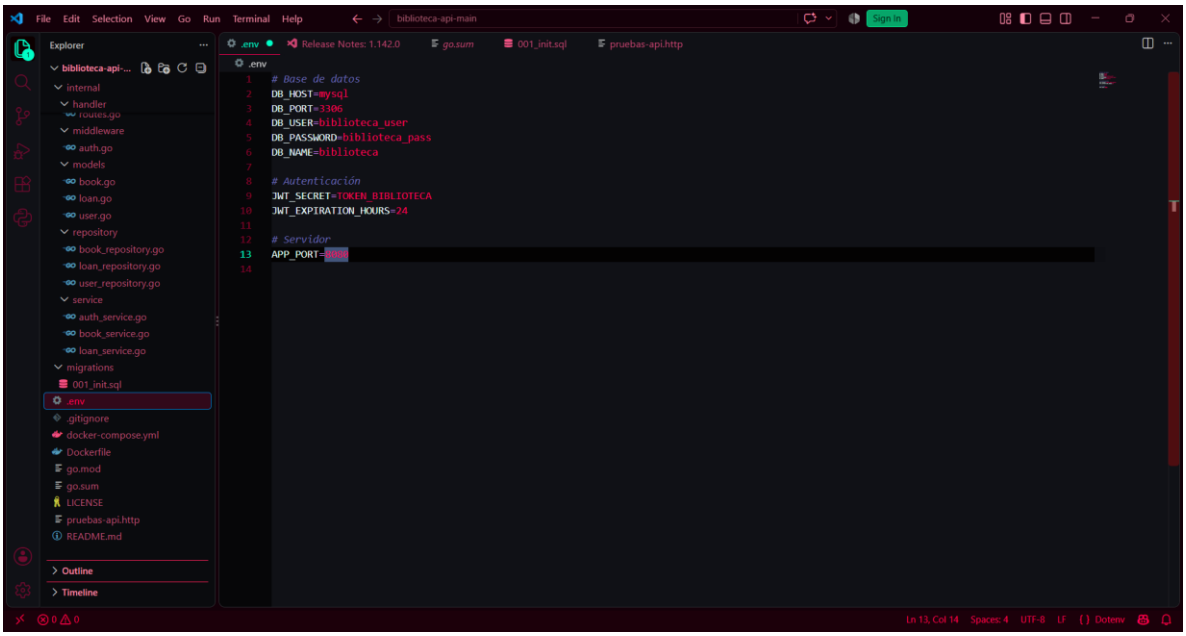
Antes de ejecutar, revisé los archivos que hacen posible levantar el proyecto con Docker:

- **Dockerfile:** es la receta para construir la imagen de la API. Es *multi-stage*: en una primera etapa parte de una imagen de Go, descarga las dependencias (go mod download) y compila el binario (go build); en una segunda etapa copia solo el binario a una imagen ligera, de modo que la imagen final no lleva el compilador ni el código fuente.
- **docker-compose.yml:** define los dos servicios del sistema, mysql (imagen mysql:8.0 de Docker Hub) y api (construida desde el Dockerfile). Los conecta en una red propia, publica los puertos 3306 y 8080, crea el volumen mysql_data para que los datos persistan, configura un *healthcheck* para MySQL y hace que la API solo arranque cuando MySQL esté sano (depends_on con condition: service_healthy).
- **.env.example:** plantilla con las variables de entorno que necesita la API. Se copia a .env, que no se sube al repositorio porque contiene credenciales (está en el .gitignore).
- **migrations/:** scripts SQL que crean las tablas y cargan datos de ejemplo. Compose monta esta carpeta en /docker-entrypoint-initdb.d del contenedor de MySQL, y MySQL los ejecuta automáticamente la primera vez que arranca con la base vacía.

Esto corresponde a la cadena vista en la Parte 1: **Dockerfile → imagen → contenedor**, con las imágenes base (golang y mysql) descargadas del **registro** (Docker Hub).

Creación del archivo .env

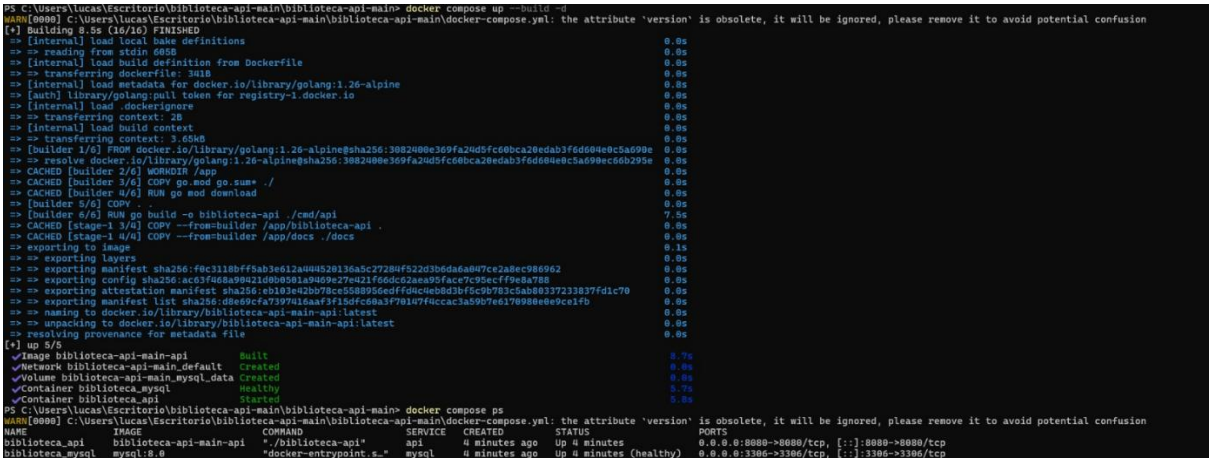
Abrí la carpeta del proyecto en visual studio. Y luego cree un archivo llamado .env. Lo siguiente que hice fue copiar el contenido de .env.example y lo pegué en el nuevo .env



Levantar los servicios

docker compose up -d

Combiné las dos opciones: --build obliga a Compose a construir la imagen de la API a partir del Dockerfile antes de arrancar, y -d (*detached*) ejecuta los contenedores en segundo plano y devuelve el control de la terminal, en lugar de quedarse mostrando los logs. Así pude seguir usando la misma terminal para la verificación.



Después arrancó primero MySQL. Cuando el *healthcheck* lo marcó como sano, MySQL ejecutó las migraciones sobre la base vacía y luego arrancó la API en el puerto 8080.

Paso 3: Verificación

Estado de los servicios

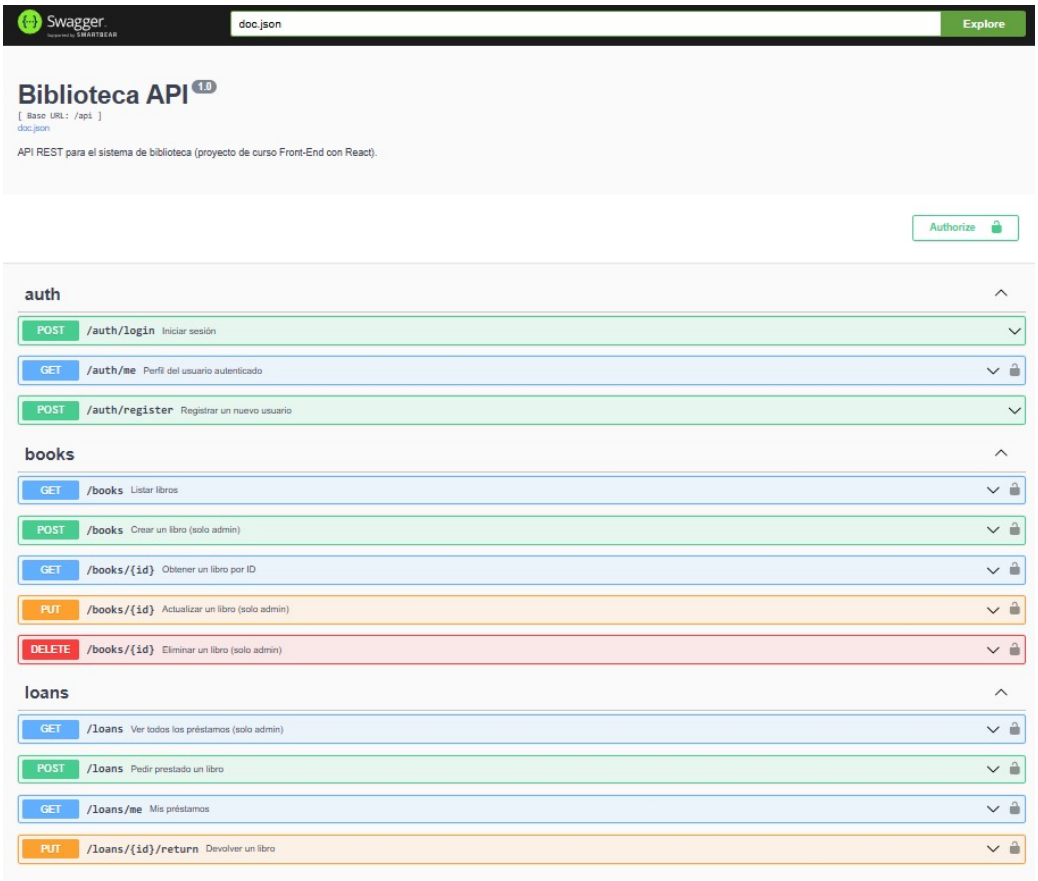
Ejecuté el comando para comprobar que los contenedores están funcionales:

docker compose ps

Los dos contenedores, biblioteca_mysql y biblioteca_api, aparecen en estado Up, y MySQL aparece como (healthy).

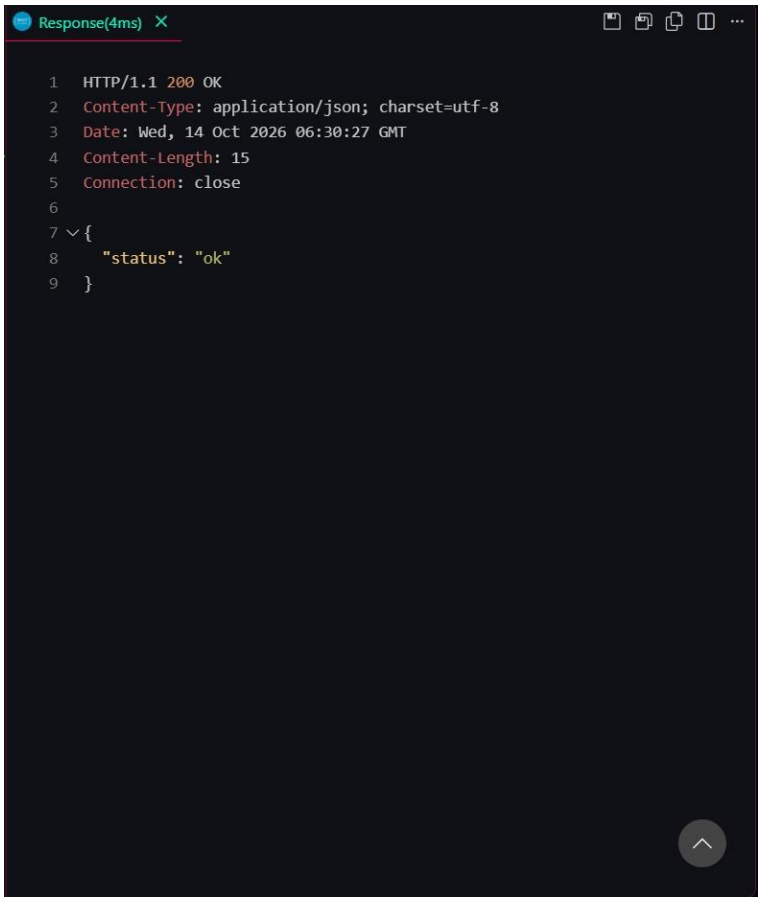
Swagger

Abrí http://localhost:8080/swagger/index.html en el navegador y cargó la documentación interactiva con los endpoints de autenticación, libros y préstamos.

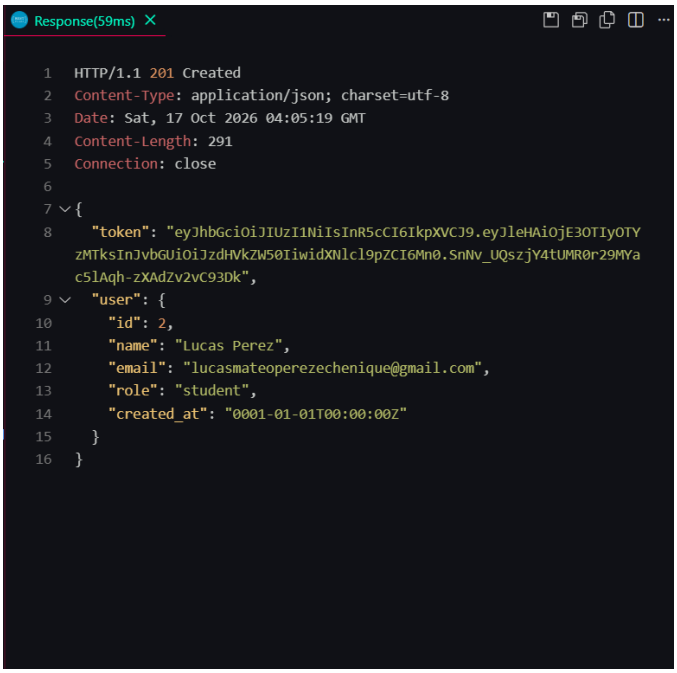


Probé el flujo completo con **[REST Client de VS Code / Swagger UI / curl]**, siguiendo la sección 3.4 c) del README.

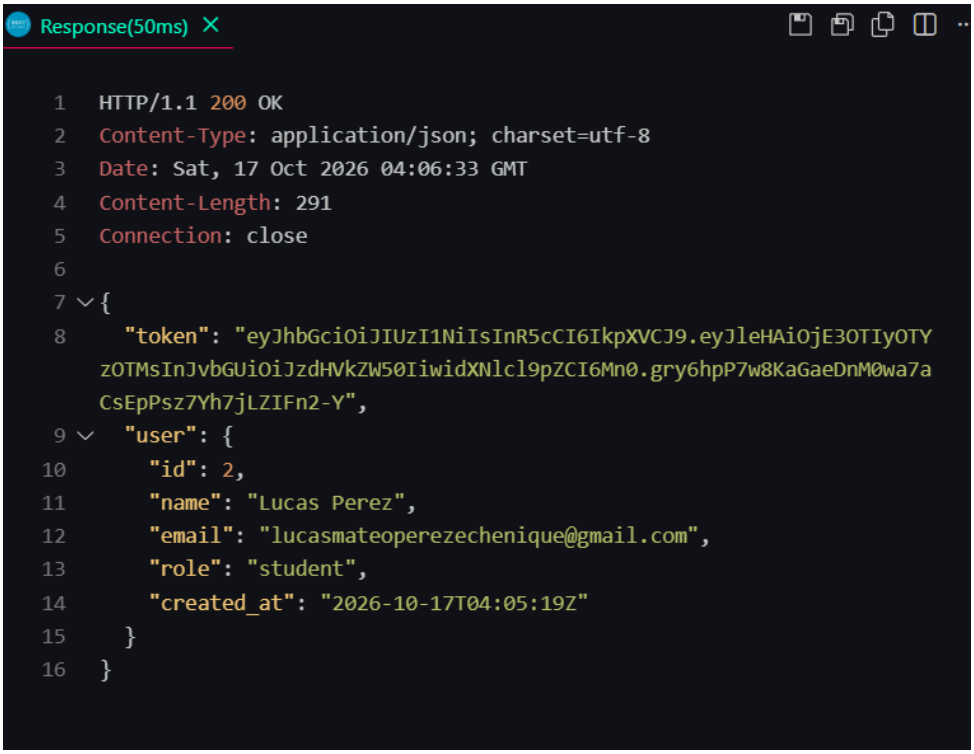
1. **Estado de la API** (GET /health): respondió {"status":"ok"}, lo que confirma que el contenedor de la API está vivo.



2. **Registro** (POST /api/auth/register): se creó el usuario con rol student y la API devolvió un token.



3. Inicio de sesión (POST /api/auth/login): con el mismo correo y contraseña, la API devolvió un token JWT nuevo.



4. Listar libros (GET /api/books con Authorization: Bearer <token>): la API devolvió los libros cargados por el seed de las migraciones. Esto demuestra que funciona toda la cadena: la API valida el token, consulta MySQL y responde.


Response(8ms)
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Date: Wed, 14 Oct 2026 06:18:58 GMT
Content-Length: 1245
Connection: close
[
{
"id": 1,
"title": "Cien años de soledad",
"author": "Gabriel García Márquez",
"isbn": "9780307474728",
"category": "Novela",
"cover_url": "",
"total_copies": 3,
"available_copies": 3,
"created_at": "2026-10-14T06:00:24Z"
},
{
"id": 2,
"title": "Clean Code",
"author": "Robert C. Martin",
"isbn": "9780132350884",
"category": "Tecnología",
"cover_url": "",
"total_copies": 2,
"available_copies": 2,
"created_at": "2026-10-14T06:00:24Z"
},
{
"id": 3,
"title": "El principito",
"author": "Antoine de Saint-Exupéry",